

Da&Da Solutions

Guia Oficial de Ingenieria Back-end

Estandares, patrones y mejores practicas para todos los servicios de Da&Da Solutions

Documento Interno — Confidencial

Version 3.1 | Ejercicio fiscal 2026 | Departamento de Ingenieria

1. Proposito y Alcance de Este Documento

Esta guia es el documento normativo de referencia para todos los ingenieros de Back-end de Da&Da Solutions, sin importar el lenguaje o el framework que usen. Establece los estandares minimos que deben satisfacerse para que un servicio sea considerado 'production-ready'. El incumplimiento de las normas aqui descritas es un motivo valido para rechazar un Pull Request.

La guia se actualiza de forma colaborativa: cualquier ingeniero puede proponer cambios mediante un ADR (Architecture Decision Record) que debe ser aprobado por al menos dos Staff Engineers antes de incorporarse a este documento. La version vigente es siempre la que esta en la rama main del repositorio dada-solutions/engineering-guidelines.

2. Filosofia de Ingenieria Back-end

2.1 Simplicidad Intencional

La complejidad accidental es el enemigo numero uno de la mantenibilidad. Antes de introducir una nueva abstraccion, patron o dependencia, debemos preguntarnos: ¿Que problema concreto resuelve esto? ¿Hay una solucion mas simple que funcione igual de bien? ¿Que pasa si necesitamos modificar esto en seis meses con un desarrollador que nunca lo vio?

El codigo mas seguro es el codigo que no existe. El mejor refactor es el que elimina lineas. La complejidad debe estar justificada por requisitos reales, no por anticipacion de escenarios hipoteticos.

2.2 APIs como Productos

Cada API que Da&Da Solutions construye es un producto con sus propios usuarios (otros servicios, el frontend, clientes externos). Esto implica que las APIs deben ser: confiables (contratos estables, versionado semantico), predecibles (comportamiento consistente y documentado), observables (metricas, logs y traces desde el dia uno) y operables (health checks, graceful shutdown, configuracion externalizada).

2.3 Operabilidad Desde el Dia Uno

Un servicio que no puede monitorearse es un servicio ciego. La observabilidad no es una caracteristica que se agrega al final: es un requisito funcional desde el primer commit. Todo servicio nuevo de Da&Da Solutions debe nacer con: logs estructurados en formato JSON, metricas expuestas en formato Prometheus, tracing con OpenTelemetry, y endpoints de health check.

3. Stack Tecnologico Aprobado

3.1 Python con FastAPI — Stack Principal para Datos e IA

FastAPI es el framework web Python preferido en Da&Da Solutions para servicios que manejan logica de negocio orientada a datos, integraciones con modelos de ML/AI, o APIs de alto rendimiento con tipado estricto. La version minima de Python soportada es 3.11 (con acceso a las mejoras de rendimiento del interprete introducidas en esa version).

Requisitos tecnicos para servicios FastAPI: usar Pydantic v2 para todos los modelos de datos de entrada y salida, usar SQLAlchemy 2.0 con soporte async para la capa de persistencia, gestionar dependencias con Poetry (el archivo pyproject.toml es la unica fuente de verdad de dependencias), y estructurar el proyecto siguiendo el patron de repositorio corporativo disponible en [dada-solutions/fastapi-template](#).

3.2 Node.js con NestJS — Stack para Logica de Negocio Compleja

NestJS es el framework Node.js preferido para servicios con logica de negocio compleja que se beneficia del sistema de modulos, inyeccion de dependencias y decoradores de NestJS. Se usa especialmente para servicios que orquestan flujos de negocio complejos con multiples dependencias.

Requisitos tecnicos para servicios NestJS: TypeScript en modo strict (tsconfig.json corporativo en el repositorio de plantillas), pnpm como gestor de paquetes, Jest para testing, y Prisma como ORM para PostgreSQL. La estructura de directorios debe seguir el patron de capas: controllers, services, repositories, y entidades de dominio.

3.3 Go — Stack para Servicios de Infra de Alta Performance

Go se usa selectivamente para servicios de infraestructura o componentes de plataforma que requieren latencias extremadamente bajas (sub-milisegundo) o concurrencia masiva: el API Gateway, los workers de procesamiento de streams de Kafka de alto throughput, y proxies o sidecars de red. El uso de Go en servicios de negocio nuevos requiere aprobacion explicita del Staff Engineer del area.

3.4 Java con Spring Boot — Legacy y Servicios de Datos Criticos

Algunos servicios heredados de la arquitectura monolitica original de Da&Da Solutions estan escritos en Java con Spring Boot. Estos servicios son mantenidos y reciben updates de seguridad, pero no se desarrollan servicios nuevos en Java sin justificacion tecnica muy fuerte. El plan de migracion a largo plazo es reescribir estos servicios incrementalmente en Python o Node.js.

4. Diseno de APIs RESTful

4.1 Principios Generales

Las APIs de Da&Da Solutions siguen el estilo arquitectonico REST con los siguientes principios adicionales: las URLs identifican recursos (sustantivos), los verbos HTTP describen las acciones, el formato de datos es JSON para todos los requests y responses, el codigo de estado HTTP comunicar el resultado de la operacion con precision, y toda la API debe poder ser documentada

en OpenAPI 3.1.

4.2 Convencion de URLs

Recursos: siempre en plural y en minusculas con palabras separadas por guiones. Ejemplos correctos: `/api/v1/users`, `/api/v1/billing-accounts`, `/api/v1/subscription-plans`. Ejemplos incorrectos: `/api/v1/getUser`, `/api/v1/User`, `/api/v1/users_list`.

Sub-recursos: `/api/v1/users/{user_id}/addresses/{address_id}`. Acciones especiales que no encajan en el modelo CRUD: `/api/v1/users/{user_id}/actions/verify-email` (usando el sufijo `/actions/{nombre-de-la-accion}`).

4.3 Versionado de API

El versionado es en la URL, por mayor version: `/api/v1/`, `/api/v2/`. Un bump de mayor version solo ocurre cuando hay un breaking change en el contrato. Cambios backward-compatible (nuevos endpoints, nuevos campos opcionales en responses, nuevos parametros opcionales en requests) no requieren cambio de version.

Cuando se deprecia una version de API, esta se mantiene activa por al menos 6 meses con un header de deprecacion en las respuestas: `Deprecation: date=2026-12-31` y `Sunset: date=2027-01-15`. Los clientes de la API deprecada reciben emails automaticos de notificacion.

4.4 Estructura de Respuesta Unificada

Todas las respuestas de la API (exitosas y de error) siguen la misma estructura envelope para garantizar coherencia: `{ success: boolean, data: T | null, error: ErrorObject | null, meta: PaginationMeta | null }`. Ejemplo de exito: `{ success: true, data: { id: 123, name: 'Ejemplo' }, error: null, meta: null }`. Ejemplo de error: `{ success: false, data: null, error: { code: USER_NOT_FOUND, message: 'El usuario 123 no fue encontrado.', details: [] }, meta: null }`.

4.5 Paginacion

Todos los endpoints que retornan colecciones implementan paginacion. Para colecciones de tamano desconocido o muy grande, se prefiere paginacion basada en cursor sobre la paginacion offset/limit (la primera es mas eficiente y consistente ante inserciones concurrentes). Los parametros estandar para offset-based son: `page` (1-indexed, default 1), `page_size` (default 20, maximo 100, configurable por endpoint). Para cursor-based: `cursor` (token opaco devuelto por el servidor), `limit` (equivalente a `page_size`). El objeto meta de la respuesta incluye: `total`, `page`, `page_size`, `next_cursor` (para cursor-based).

4.6 Filtrado, Búsqueda y Ordenamiento

Los parametros de filtrado van como query params. La convencion es usar el nombre del campo directamente para filtros de igualdad exacta: `/api/v1/orders?status=pending`. Para filtros de rango: `created_at_gte` y `created_at_lte`. Para búsqueda de texto libre: `q=termino-de-busqueda`. El ordenamiento usa `sort_by` (nombre del campo) y `sort_order` (asc o desc, default asc).

5. Autenticacion y Autorizacion

5.1 Autenticacion con JWT

Da&Da Solutions usa JWT (RFC 7519) para la autenticacion de todos los endpoints publicos. El auth-service es el unico emisor de tokens en la plataforma. Los tokens siguen el esquema de Access Token + Refresh Token: el Access Token tiene TTL de 15 minutos y es stateless; el Refresh Token tiene TTL de 7 dias, se almacena en una cookie HttpOnly Secure SameSite=Strict, y es stateful (guardado en Redis con posibilidad de revocacion).

El Access Token es un JWT firmado con RS256 (RSA 2048 bits). La clave privada vive exclusivamente en el auth-service; todos los demas servicios validan tokens usando la clave publica descargada del JWKS endpoint del auth-service. Nunca se comparten claves privadas.

5.2 Control de Acceso Basado en Roles (RBAC)

La autorizacion se implementa en dos niveles. El primero es en el API Gateway (verificacion de que el token es valido y no ha expirado). El segundo es en cada servicio individual (verificacion de que el usuario tiene los permisos necesarios para la operacion especifica). Los roles globales son: SUPER_ADMIN, ADMIN, MANAGER, OPERATOR y VIEWER. Adicionalmente, existe un sistema de permisos granulares a nivel de recurso para casos complejos.

5.3 Comunicacion Servicio-a-Servicio

La comunicacion entre microservicios internos usa Mutual TLS (mTLS) gestionado por Istio para cifrado y autenticacion de identidad. Para autorizacion de acciones especificas entre servicios (no solo autenticacion de identidad), cada servicio emite un Service Account Token con permisos especificos que el servicio receptor valida.

6. Manejo de Errores

6.1 Principios Generales del Manejo de Errores

Los errores son ciudadanos de primera clase en el codigo de Da&Da Solutions. Principios: los errores deben ser explicitamente tipados y manejados, nunca silenciados; las excepciones que se capturan deben loggarse con contexto suficiente para el debugging en produccion; los errores de negocio (validacion, permisos, recursos no encontrados) se retornan al cliente con informacion util; los errores de sistema (fallos de infraestructura) se retornan como HTTP 500 con un mensaje generico (los detalles van a los logs, no al cliente).

6.2 Catalogo de Error Codes

Cada dominio de negocio tiene un catalogo de error codes con el formato DOMINIO_ENTIDAD_RAZON. Algunos ejemplos estandarizados: AUTH_TOKEN_EXPIRED, AUTH_TOKEN_INVALID, AUTH_INSUFFICIENT_PERMISSIONS, USER_NOT_FOUND, USER_EMAIL_ALREADY_EXISTS, USER_ACCOUNT_SUSPENDED, BILLING_PAYMENT_FAILED, BILLING_INSUFFICIENT_FUNDS, BILLING_SUBSCRIPTION_EXPIRED, ORDER_ITEM_OUT_OF_STOCK. El catalogo completo y

las guías para agregar nuevos error codes estan en Confluence.

6.3 Excepciones No Manejadas

Todo servicio debe tener un handler de excepciones global que: loggee la excepcion completa con su stack trace y el contexto de la request (sin datos PII), retorne una respuesta HTTP 500 con un `error_code` generico de `SERVER_UNEXPECTED_ERROR`, genere una alerta en PagerDuty si la tasa de errores 500 supera el umbral configurado, e incluya el `trace_id` en la respuesta para que el cliente pueda reportarlo al soporte.

7. Testing — Piramide y Estandares

7.1 La Piramide de Tests de Da&Da; Solutions

Seguimos la piramide de testing clasica con proporciones especificas: 70% tests unitarios, 20% tests de integracion, 10% tests end-to-end. La cobertura minima de tests unitarios para poder mergear a main es del 80%. Esta cifra se verifica automaticamente en el pipeline de CI y un resultado menor bloquea el merge.

7.2 Tests Unitarios

Los tests unitarios prueban una unidad logica de codigo (generalmente una funcion o clase) en completo aislamiento de sus dependencias externas. Las dependencias externas (bases de datos, APIs externas, servicios externos) siempre se mockean. En Python usamos `pytest` con `pytest-asyncio` para funciones `async` y `unittest.mock` para mocks. En Node.js usamos Jest con los mocks nativos de Jest.

Convenciones de naming: `test_should_{expected_behavior}_when_{condition}` para Python, `should {expected behavior} when {condition}` para Jest. Estructura de cada test: Arrange (preparar el estado), Act (ejecutar la accion bajo prueba), Assert (verificar el resultado esperado).

7.3 Tests de Integracion

Los tests de integracion verifican la interaccion entre multiples componentes del sistema (por ejemplo, el handler HTTP, la capa de servicio y la capa de repositorio con una base de datos real). En Da&Da Solutions usamos `testcontainers` para levantar instancias efimeras de PostgreSQL, MongoDB o Redis directamente en el test. Los tests de integracion son mas lentos que los unitarios pero mas representativos de la realidad.

7.4 Contract Testing con Pact

Para los servicios que se comunican entre si via REST, usamos Pact para contract testing. El Consumer define el contrato de lo que espera de la API del Provider y lo comparte en el Pact Broker. El Provider verifica automaticamente que cumple todos los contratos de sus consumers como parte de su pipeline de CI. Esto previene silenciosamente que cambios en un servicio rompan a sus clientes.

7.5 Tests de Performance

Los servicios criticos tienen pruebas de performance con k6. Estas pruebas simulan la carga esperada en produccion y verifican que los SLOs de latencia (P99 menor a 500ms) se mantengan. Se ejecutan en el entorno de staging previo a cada release mayor.

8. Observabilidad — Los Tres Pilares

8.1 Logs Estructurados

Todos los servicios deben generar logs en formato JSON estructurado. El formato estandar de cada evento de log incluye obligatoriamente: timestamp (ISO 8601 con zona horaria UTC), level (DEBUG, INFO, WARNING, ERROR, CRITICAL), service (nombre del microservicio), version (version del servicio), trace_id (ID de la traza distribuida), span_id (ID del span actual), y message (descripcion del evento). En Python usamos structlog configurado con el procesador JSON. En Node.js usamos pino.

Reglas criticas sobre el contenido de los logs: nunca loggear datos PII (nombres, emails, numeros de tarjeta, passwords) directamente — solo loggear identificadores opacos como IDs de usuario; no loggear en nivel DEBUG en produccion; loggear siempre el trace_id para permitir correlacion de logs entre servicios.

8.2 Metricas con Prometheus

Cada servicio expone un endpoint GET /metrics en formato Prometheus. Las metricas minimas requeridas son: http_request_duration_seconds (histograma con labels: method, path, status_code), http_requests_total (counter con labels: method, path, status_code), active_connections (gauge), y metricas del proceso (CPU, memoria, goroutines/threads). En Python usamos prometheus-client. En Node.js usamos prom-client.

8.3 Tracing Distribuido con OpenTelemetry

El tracing distribuido es esencial para diagnosticar problemas de latencia y errores en sistemas de microservicios. Cada servicio debe instrumentarse con los SDKs de OpenTelemetry para su lenguaje. El trace_id se propaga automaticamente en los headers HTTP usando el formato W3C Trace Context (traceparent y tracestate). Los traces se exportan al backend de observabilidad (Jaeger en desarrollo/staging, Grafana Tempo en produccion) via OTLP.

8.4 Health Checks

Cada servicio implementa dos endpoints de health: GET /health/liveness retorna 200 OK si el proceso esta vivo (no esta en deadlock ni en estado corrupto). GET /health/readiness retorna 200 OK si el servicio esta listo para manejar trafico (la conexion a la base de datos esta activa, las dependencias criticas estan disponibles). Kubernetes usa liveness para reiniciar pods en mal estado y readiness para enrutar trafico solo a pods sanos.

9. Seguridad

9.1 OWASP Top 10 — Consideraciones para Da&Da;

Los riesgos del OWASP Top 10 mas relevantes para el stack de Da&Da Solutions son: Injection (usar siempre ORM o queries parametrizadas, nunca concatenar inputs de usuario en SQL), Broken Access Control (verificar autorizacion en cada endpoint, nunca asumir que si el request llega con un token valido tiene acceso a todos los recursos), Cryptographic Failures (usar bcrypt con factor de costo minimo 12 para passwords, AES-256 para datos en reposo, TLS 1.2 o superior para datos en transito), y Security Misconfiguration (no exponer informacion sensible en mensajes de error, no tener servicios innecesarios expuestos, rotar secretos regularmente).

9.2 SAST y SCA en el Pipeline de CI

El pipeline de CI incluye analisis de seguridad estatico (SAST) con Bandit (Python) y Semgrep (multi-lenguaje). Tambien incluye analisis de composicion de software (SCA) con Safety (Python) y npm audit (Node.js) para detectar dependencias con vulnerabilidades conocidas. Un hallazgo de severidad HIGH o CRITICAL bloquea el merge hasta ser resuelto o marcado explicitamente como falso positivo con justificacion documentada.

9.3 Rotacion de Secretos

Ningun secreto es permanente. Los secretos de servicio (API keys de terceros, claves de cifrado, certificados) se rotan al menos cada 90 dias. Las credenciales de base de datos se rotan diariamente usando las dynamic secrets de HashiCorp Vault. El proceso de rotacion es automatico y los servicios deben ser capaces de recibir nuevas credenciales sin reiniciarse (Vault Agent Sidecar inyecta las credenciales nuevas automaticamente).

10. Performance y Optimizacion

10.1 Async por Defecto

Todos los handlers HTTP en FastAPI y NestJS deben ser async/await para no bloquear el event loop durante operaciones de I/O. Una funcion sincrona que realiza una operacion de I/O (query de base de datos, llamada HTTP) en un handler que corre en el event loop es un bug de performance grave que puede degradar la capacidad de la aplicacion a medida que aumenta la concurrencia.

10.2 Optimizacion de Queries de Base de Datos

El problema N+1 es la causa mas comun de degradacion de performance en aplicaciones back-end. Para detectarlo y prevenirlo: usar el modo de logging de SQLAlchemy o el query logger de Prisma en desarrollo para ver todas las queries ejecutadas, configurar alertas en Grafana para queries que tarden mas de 100ms en promedio, y revisar los planes de ejecucion con EXPLAIN ANALYZE para las queries mas frecuentes y criticas.

Reglas para indices: crear indices en todos los campos usados en clausulas WHERE, JOIN ON y ORDER BY, usar indices compuestos para patrones de query con multiples columnas en la clausula WHERE, evitar indices de baja cardinalidad (ej. un campo boolean con el 90% de

valores iguales), y crear indices en produccion siempre con `CREATE INDEX CONCURRENTLY` (para no bloquear la tabla durante la creacion).

10.3 Gestion de Connection Pools

La conexion a bases de datos debe gestionarse mediante connection pools para evitar el overhead de crear una nueva conexion TCP para cada request. Los parametros minimos de configuracion del pool son: `max_connections` (numero maximo de conexiones activas, tipicamente igual al numero de workers del servicio), `min_connections` (conexiones minimas mantenidas abiertas), y `connection_timeout` (tiempo maximo de espera para obtener una conexion del pool). Valores fuera de rango en estos parametros son una causa frecuente de problemas bajo carga.

11. Releases y Despliegues

11.1 Semantic Versioning

Todos los servicios de Da&Da Solutions siguen el versionado semantico SemVer (MAJOR.MINOR.PATCH). MAJOR: cambios incompatibles con versiones anteriores de la API publica. MINOR: nueva funcionalidad backward-compatible. PATCH: correcciones de bugs backward-compatible. Los numeros de version se calculan y se aplican automaticamente por el pipeline de CD usando la herramienta semantic-release y los mensajes de commit en formato Conventional Commits.

11.2 Conventional Commits

Da&Da Solutions aplica el estandar de Conventional Commits para todos los mensajes de commit. El formato es: `tipo(alcance): descripcion corta`. Tipos validos: `feat` (nueva funcionalidad, incrementa MINOR), `fix` (correccion de bug, incrementa PATCH), `docs` (cambios en documentacion), `style` (formateado de codigo, no afecta logica), `refactor` (refactorizacion sin cambio de funcionalidad), `test` (agregar o modificar tests), `chore` (cambios en herramientas, dependencias o configuracion de CI/CD). Un `BREAKING CHANGE` en el cuerpo o footer del commit incrementa MAJOR.

12. Migraciones de Base de Datos

12.1 Principios de Migraciones Seguras

Las migraciones de base de datos son una de las operaciones de mayor riesgo en produccion. En Da&Da Solutions seguimos el patron de migraciones backward-compatible para poder hacer rollback en cualquier momento sin perdida de datos. Esto implica que nunca se hace un cambio destructivo (`DROP TABLE`, `DROP COLUMN`) en la misma migracion en que se despliega el nuevo codigo que deja de usar esa estructura.

El proceso para un cambio destructivo seguro tiene tres fases: Fase 1: agregar la nueva estructura sin eliminar la vieja y deployar el código compatible con ambas. Fase 2: migrar los datos a la nueva estructura. Fase 3: una vez verificado que el código nuevo funciona correctamente en producción por al menos 24 horas, eliminar la estructura vieja en una migración separada.

12.2 Herramientas de Migración

Para Python/FastAPI con SQLAlchemy usamos Alembic. Cada migración es un archivo Python con funciones `upgrade()` y `downgrade()` que deben ser idempotentes cuando sea posible y ejecutarse en menos de 30 segundos en producción. Para Node.js/NestJS con Prisma usamos Prisma Migrate. El SQL generado automáticamente debe revisarse siempre antes de commitear y se aplica en producción vía `prisma migrate deploy` en el pipeline de CD.

12.3 Índices y Performance en Migraciones

Agregar un índice a una tabla grande en producción con PostgreSQL puede bloquear las escrituras durante varios minutos si no se hace correctamente. La regla es siempre usar `CREATE INDEX CONCURRENTLY` para agregar índices en producción. Alembic no soporta esta opción de forma nativa, por lo que se debe usar SQL explícito en la migración desactivando las transacciones en la función `upgrade()`.

13. Mensajería Asíncrona con Kafka

13.1 Cuando Usar Kafka vs REST Sincrónico

Kafka es la opción correcta cuando: el productor y el consumidor no necesitan conocerse ni estar disponibles al mismo tiempo, la operación puede fallar y se necesita reintento automático, el mismo evento necesita ser consumido por múltiples servicios (fan-out), o se necesita un registro inmutable y reproducible de todos los eventos del dominio. REST sincrónico es preferible cuando el cliente necesita saber el resultado de la operación inmediatamente para continuar, o cuando la latencia de extremo a extremo es crítica.

13.2 Convención de Nombres de Topics

Los nombres de topics de Kafka en Da&Da Solutions siguen la convención: `{dominio}.{entidad}.{evento}` en formato kebab-case. Ejemplos: `billing.subscription.created`, `user.account.email-verified`, `payment.transaction.failed`. Los nombres de topics son permanentes una vez creados en producción; cambiarlos requiere un proceso de migración complejo.

13.3 Schema Registry y Contrato de Mensajes

Todos los mensajes publicados a Kafka deben tener un schema registrado en Confluent Schema Registry. El formato es Avro para mensajes internos y JSON Schema para mensajes externos. Los schemas siguen reglas de compatibilidad BACKWARD: agregar campos opcionales es compatible, eliminar campos no lo es. Los cambios que rompen la compatibilidad requieren un proceso de doble publicación durante el periodo de migración.

14. Patrones de Diseño Aplicados

14.1 Repository Pattern

Da&Da Solutions usa el Repository Pattern para aislar la lógica de acceso a datos de la lógica de negocio. Cada entidad de dominio tiene una interfaz de repositorio que define las operaciones de acceso a datos y una implementación concreta que usa SQLAlchemy o Prisma. La capa de servicio siempre interactúa con la interfaz, nunca con el ORM directamente.

14.2 CQRS (Command Query Responsibility Segregation)

Para servicios de alta carga con patrones de lectura muy diferentes a los de escritura, Da&Da Solutions aplica CQRS de manera pragmática: modelos de datos separados para Commands y Queries, optimizando cada uno para su caso de uso. El modelo de escritura puede usar un schema normalizado óptimo para transacciones, mientras que el modelo de lectura puede usar vistas materializadas u otras bases de datos (Redis, Elasticsearch).

14.3 Circuit Breaker para Resiliencia

Los servicios que llaman a APIs externas o a otros microservicios deben implementar el patrón Circuit Breaker para prevenir fallos en cascada. Usamos tenacity en Python y opossum en Node.js. El Circuit Breaker tiene tres estados: Closed (funcionando normalmente), Open (fallos detectados, todas las llamadas fallan rápido), y Half-Open (periodo de prueba para verificar recuperación).

14.4 Outbox Pattern para Consistencia Eventual

Cuando una operación debe actualizar la base de datos y publicar un evento a Kafka de manera atómica, el patrón Outbox garantiza la consistencia. Dentro de la misma transacción de base de datos, se inserta el evento a publicar en una tabla auxiliar (outbox). Un proceso separado (outbox processor) lee periódicamente la tabla y publica los eventos pendientes a Kafka, marcándolos como procesados.

15. Estándares de Documentación de Código

15.1 Docstrings y Comentarios

En Da&Da Solutions el código debe ser autoexplicativo en la mayoría de los casos. Los comentarios deben explicar el 'por qué', no el 'qué'. Un comentario que dice 'incrementa el contador en 1' no agrega valor; un comentario que dice 'se incrementa aquí y no en el handler porque necesitamos contar intentos fallidos antes de la autenticación' sí lo hace.

Los docstrings son obligatorios en: todas las funciones públicas de módulos de librería, todos los métodos de clases de dominio y repositorios, todos los endpoints de API (el docstring se convierte en la descripción del endpoint en la documentación OpenAPI generada).

automaticamente), y todos los modulos que implementan logica de negocio no trivial.

15.2 Generacion Automatica de Documentacion de API

FastAPI genera automaticamente la documentacion OpenAPI 3.1 a partir de los type hints de Python y los docstrings de los endpoint handlers. Para que esta documentacion sea util, cada endpoint debe tener: un docstring con la descripcion funcional del endpoint, modelos Pydantic con Field() descriptivo para cada campo, response_model explicito para cada endpoint, y ejemplos representativos en los modelos.

La documentacion Swagger generada automaticamente es accesible en /docs en todos los ambientes (excepto produccion donde esta restringida a IPs corporativas). La version Redoc esta disponible en /redoc.

15.3 Architecture Decision Records (ADRs)

Cuando el equipo toma una decision tecnica significativa (eleccion de una libreria, cambio de patron, adopcion de una nueva tecnologia), se debe escribir un ADR. El formato incluye: Titulo, Fecha, Estado (Propuesto / Aceptado / Deprecado), Contexto, Decision, Alternativas consideradas (con pros y contras), Consecuencias, y Notas.

Los ADRs se almacenan en el directorio docs/adr/ del repositorio con nombres en el formato NNNN-titulo-en-kebab-case.md. Una vez aceptado, un ADR nunca se modifica; si la decision cambia, se crea un nuevo ADR que indica que reemplaza al anterior.

Anexo A: Políticas de Calidad y Mejora Continua

A.1 Propósito y Alcance

En Da&Da Solutions, la calidad no es una opción, es un pilar fundamental de nuestra entrega de valor. Este anexo establece las bases de nuestro Sistema de Gestión de Calidad (SGC) aplicable a todas las áreas de ingeniería, desarrollo, operaciones y soporte de la organización. Nuestro compromiso es mantener estándares de excelencia en todos los productos y servicios que desarrollamos, asegurando la satisfacción de nuestros clientes y el cumplimiento de los requisitos legales y regulatorios vigentes en todos los mercados donde operamos.

El alcance de esta política abarca desde la concepción y diseño de los sistemas, pasando por el desarrollo, pruebas, despliegue, hasta el mantenimiento y soporte continuo. Todo el personal interno, así como contratistas y proveedores de servicios de Da&Da Solutions, están obligados a conocer, comprender y aplicar estos principios en su labor diaria.

A.2 Principios de Calidad

Nuestros principios de calidad se centran en:

- **Enfoque al Cliente:** Entender las necesidades actuales y futuras de nuestros clientes para superar sus expectativas.
- **Liderazgo:** Los líderes de todos los niveles deben establecer un propósito claro y crear un entorno interno que promueva la participación activa del personal.
- **Compromiso del Personal:** La competencia, empoderamiento y compromiso de las personas en todos los niveles son esenciales para crear y entregar valor.
- **Enfoque Basado en Procesos:** Los resultados consistentes se logran de manera más efectiva cuando las actividades se entienden y gestionan como procesos interrelacionados.
- **Mejora Continua:** La mejora constante del desempeño general de la organización es un objetivo permanente.
- **Toma de Decisiones Basada en Evidencia:** Las decisiones efectivas se basan en el análisis de datos y la información.
- **Gestión de Relaciones:** Fomentar relaciones mutuamente beneficiosas con proveedores y socios estratégicos.

A.3 Métricas e Indicadores de Calidad

Para asegurar el cumplimiento de nuestros objetivos de calidad, Da&Da Solutions monitoriza de forma continua un conjunto de indicadores clave de rendimiento (KPIs). Estos incluyen:

- **Defect Escape Rate (DER):** Porcentaje de defectos que se escapan a producción frente al total de defectos encontrados.
- **Time to Resolve (TTR):** Tiempo promedio necesario para resolver incidentes críticos reportados por clientes.

- **Customer Satisfaction Score (CSAT):** Indice de satisfaccion del cliente recopilado mediante encuestas periodicas tras la resolucion de tickets y entrega de proyectos.
- **Code Coverage:** Porcentaje de codigo cubierto por pruebas automatizadas (unitarias, integracion, e2e). El objetivo minimo es del 85% para todos los proyectos nuevos.
- **Technical Debt Ratio:** Relacion entre el costo de solucionar problemas estructurales del codigo y el costo de desarrollar nuevo software.

A.4 Auditorias Internas y Externas

Da&Da Solutions realiza auditorias internas trimestrales para verificar el cumplimiento de los procesos documentados y detectar oportunidades de mejora. Estas auditorias son lideradas por el equipo de QA Corporativo, y sus resultados se presentan a la direccion ejecutiva para su revision. Ademas, nos sometemos a auditorias externas anuales para mantener nuestras certificaciones internacionales (como ISO 9001 e ISO 27001). Todo colaborador debe cooperar activamente y con total transparencia durante cualquier proceso de auditoria.

Anexo B: Normativas de Compliance y Etica Corporativa

B.1Codigo de Conducta y Etica

ElCodigo de Conducta de Da&Da Solutions es la brujula que guia nuestras acciones y decisiones diarias. Esperamos que todos los empleados, contratistas y colaboradores actuen con la maxima integridad, honestidad y respeto en todas sus interacciones, ya sea con companeros de trabajo, clientes, socios o competidores.

No toleramos ninguna forma de discriminacion, acoso (sexual, laboral o de cualquier otro tipo), ni represalias contra quienes reporten infracciones de buena fe. Promovemos un ambiente de trabajo inclusivo y diverso donde todas las voces son valoradas.

B.2Prevencion del Fraude y Corrupcion

Da&Da Solutions se rige por politicas estrictas de tolerancia cero ante el soborno y la corrupcion. Esta terminantemente prohibido ofrecer, prometer, dar, aceptar o solicitar cualquier ventaja indebida (financiera o de otro tipo) para obtener un beneficio comercial o personal. Esto incluye regalos extravagantes, atenciones desproporcionadas y pagos de facilitacion.

Cualquier colaborador que tenga conocimiento o sospecha de un acto de corrupcion debe reportarlo inmediatamente a traves del Canal de Denuncias Eticas, el cual garantiza el anonimato y la confidencialidad absoluta.

B.3Conflictos de Interes

Un conflicto de interes ocurre cuando los intereses personales de un colaborador interfieren, o parecen interferir, con los intereses de Da&Da Solutions. Todos los empleados tienen la obligacion de evitar situaciones en las que su juicio objetivo pueda verse comprometido.

Ejemplos comunes de conflictos de interes incluyen:

- Tener una relacion financiera o comercial con un proveedor, cliente o competidor.
- Usar informacion confidencial de la empresa para beneficio personal (Insider Trading).
- Contratar o supervisar directamente a un familiar cercano o pareja sentimental.
- Trabajar en proyectos paralelos o consultar para empresas de la misma industria sin autorizacion previa.

Cualquier posible conflicto de interes debe ser revelado de inmediato al departamento de RRHH y al manager directo para su evaluacion y gestion adecuada.

B.4 Confidencialidad y Propiedad Intelectual

Toda la informacion generada o procesada dentro de Da&Da Solutions es propiedad exclusiva de la empresa o de sus clientes. Esto abarca codigo fuente, algoritmos, disenos, estrategias de negocio, bases de datos y cualquier documentacion tecnica o comercial.

Los colaboradores tienen el deber de mantener la mas estricta confidencialidad sobre esta informacion, incluso despues de finalizada su relacion laboral con la empresa. Asimismo, todo desarrollo intelectual creado durante el horario laboral o utilizando recursos de la compania pertenece enteramente a Da&Da Solutions. Se prohíbe el uso de software sin licencia o material con derechos de autor sin la debida autorizacion.

Anexo C: Plan de Continuidad del Negocio (BCP)

C.1 Objetivos del BCP

El Plan de Continuidad del Negocio de Da&Da Solutions tiene como objetivo asegurar la operacion ininterrumpida de nuestros servicios criticos frente a eventos adversos (desastres naturales, pandemias, ciberataques a gran escala, fallas catastroficas de infraestructura). El BCP garantiza que podamos mantener nuestros compromisos contractuales, minimizar las perdidas financieras y proteger la reputacion de la compania.

C.2 Evaluacion de Riesgos y BIA (Business Impact Analysis)

De forma anual, el Comité de Riesgos realiza un Analisis de Impacto al Negocio (BIA) para identificar los procesos criticos, evaluar las consecuencias de su interrupcion y determinar los Tiempos Objetivo de Recuperacion (RTO) y los Puntos Objetivo de Recuperacion (RPO) para cada sistema y servicio de la organizacion. Los resultados del BIA dictan las estrategias de recuperacion y la asignacion de recursos para el BCP.

C.3 Equipos de Respuesta y Roles

El BCP define una estructura clara de respuesta ante crisis:

- **Comite de Manejo de Crisis (CMC):** Formado por la alta direccion. Encargado de tomar decisiones estrategicas, aprobar comunicaciones corporativas y declarar formalmente el estado de emergencia.
- **Equipo de Respuesta a Incidentes (IRT):** Formado por lideres tecnicos y de operaciones. Responsable de ejecutar las tareas tacticas para contener y mitigar el incidente en tiempo real.
- **Coordinadores de Continuidad:** Designados en cada departamento para asegurar la ejecucion ordenada del plan a nivel operativo y gestionar al personal.

C.4 Procedimientos de Evacuacion y Trabajo Remoto

En caso de que las instalaciones fisicas de Da&Da Solutions queden inutilizables, se activara el protocolo de trabajo 100% remoto para todo el personal. La infraestructura en la nube esta disenada para soportar el acceso remoto seguro (via VPN corporativa y Zero Trust Network Access) del total de la plantilla sin degradacion del servicio.

Se requiere que todos los empleados mantengan sus equipos portatiles actualizados y participen en los simulacros de evacuacion y pruebas de conectividad remota que se realizan semestralmente.

Anexo D: Responsabilidad Social Corporativa y Sostenibilidad

D.1 Compromiso Ambiental

En Da&Da Solutions reconocemos nuestra responsabilidad con el medio ambiente. Estamos comprometidos a minimizar nuestra huella de carbono mediante la optimización de recursos y la implementación de prácticas sostenibles. Nuestros objetivos ambientales para los próximos cinco años incluyen:

- Reducir el consumo energético en nuestras oficinas físicas en un 25% mediante políticas de uso eficiente e iluminación inteligente.
- Migrar el 100% de nuestra carga de trabajo en la nube a regiones de proveedores (como AWS o GCP) que se alimenten exclusivamente de energías renovables.
- Implementar un programa corporativo de reciclaje y gestión de residuos electrónicos (e-waste), asegurando que los equipos obsoletos sean donados o reciclados correctamente.

D.2 Impacto Comunitario y Voluntariado

Creemos firmemente en devolver valor a las comunidades en las que operamos. Da&Da Solutions fomenta la participación activa de sus colaboradores en programas de impacto social. Cada empleado dispone de 24 horas anuales remuneradas (3 días hábiles) para dedicarse a actividades de voluntariado en fundaciones u ONGs asociadas a la empresa.

Nuestros programas se centran principalmente en la educación tecnológica, ofreciendo mentoría y talleres de programación gratuitos a jóvenes de sectores vulnerables para reducir la brecha digital.

D.3 Diversidad e Inclusion en Tecnología

La diversidad de perspectivas es el motor de la innovación. Da&Da Solutions tiene un fuerte compromiso con la diversidad e inclusión (D&I). Nos esforzamos por crear equipos donde convivan diferentes géneros, orígenes étnicos, edades y trayectorias profesionales. Contamos con ERGs (Employee Resource Groups) activos para mujeres en tecnología, comunidad LGBTQ+, y neurodiversidad. El equipo de adquisición de talento tiene metas específicas para garantizar que los procesos de selección sean equitativos y libres de sesgos inconscientes.

Anexo E: Marco Metodologico y Adopcion Agile

E.1 Manifiesto Agile Interno

Da&Da Solutions abraza completamente los principios del desarrollo agil de software, adaptandolos a nuestra cultura y necesidades. Entendemos "Agile" no como un conjunto estricto de reuniones, sino como una mentalidad de adaptacion continua, entrega de valor temprano y colaboracion estrecha. Valoramos el software funcionando por encima de la documentacion exhaustiva (aunque, como se detalla en otros apartados, la documentacion justa y necesaria es vital).

E.2 Ciclos de Entrega (Sprints y Kanban)

Los equipos de producto suelen operar en Sprints de dos semanas utilizando metodologias Scrum, con ceremonias de Planning, Daily Stand-up, Review y Retrospectiva. Sin embargo, los equipos de operaciones e infraestructura, asi como los squads que manejan alto volumen de incidencias y soporte no planificado, utilizan Kanban para gestionar su flujo de trabajo, priorizando la reduccion del Work In Progress (WIP) y el Lead Time.

La eleccion del marco especifico se delega a la autonomia de cada equipo, siempre y cuando se cumplan los objetivos de visibilidad (a traves de Jira) y sincronizacion con el resto de la tribu.

E.3 Definition of Ready (DoR) y Definition of Done (DoD)

Para mantener la calidad y predictibilidad, aplicamos de forma estricta los conceptos de DoR y DoD:

- **Definition of Ready (DoR):** Una historia de usuario o tarea no puede entrar a un Sprint o a la columna "In Progress" si no tiene: Criterios de aceptacion claros, dependencias identificadas y resueltas, disenio UX/UI aprobado (si aplica), y ha sido refinada y estimada por el equipo.
- **Definition of Done (DoD):** Un ticket no se considera terminado hasta que: El codigo compila y pasa todos los tests (unitarios y e2e), ha sido revisado (aprobado en PR por al menos 2 pares), ha sido desplegado en el entorno de Staging y validado funcionalmente, la documentacion tecnica esta actualizada, y no introduce nuevas vulnerabilidades (seguridad).

E.4 Ceremonias de Sincronizacion a Nivel Organizacional

A nivel escalado, realizamos ceremonias para alinear a los distintos squads y tribes:

- **Sprint Demo Tribe:** Sesion mensual donde cada equipo presenta el valor entregado y demos en vivo a los stakeholders.
- **Tech Townhall:** Reunion trimestral liderada por el CTO para compartir la vision tecnologica, cambios de arquitectura globales y celebrar logros del equipo de ingenieria.
- **Post-Mortems Transversales:** Cuando un incidente severo afecta a multiples equipos, se realiza un post-mortem conjunto para asegurar que los aprendizajes se distribuyan en toda la organizacion.

Anexo F: Uso Aceptable de Tecnologías y Sistemas

F.1 Dispositivos Corporativos

Los equipos proporcionados por Da&Da Solutions (laptops, teléfonos móviles, monitores) son propiedad de la empresa y se otorgan para uso exclusivamente profesional. Se permite un uso personal incidental siempre que no interfiera con las responsabilidades laborales, no infrinja las políticas de seguridad ni consuma recursos excesivos. Esta prohibida la instalación de software pirata, herramientas de hacking (salvo autorización expresa para equipos de seguridad), mineros de criptomonedas, y clientes P2P para descarga de contenido protegido por derechos de autor.

F.2 Redes Sociales y Comunicación Pública

Alentamos a nuestros colaboradores a ser embajadores de la marca Da&Da Solutions en redes sociales (como LinkedIn, Twitter, blogs técnicos). Sin embargo, deben dejar claro que sus opiniones son personales y no representan posturas oficiales de la compañía, salvo que sean portavoces autorizados. Esta estrictamente prohibido compartir información confidencial sobre clientes, vulnerabilidades de sistemas, estrategias financieras o conflictos internos en plataformas públicas.

F.3 Uso de Herramientas de Inteligencia Artificial (IA)

El uso de herramientas de IA generativa (como ChatGPT, GitHub Copilot, Gemini) está permitido y fomentado para aumentar la productividad, siempre y cuando se adhiera a las siguientes restricciones de seguridad: No se puede ingresar código fuente propietario, credenciales, secretos, datos de clientes o información confidencial de Da&Da Solutions en versiones gratuitas o públicas de estas herramientas, ya que esos datos podrían ser utilizados para entrenar modelos externos. Se deben utilizar únicamente las licencias empresariales provistas por la compañía que garantizan la privacidad de los datos.